Sockets TCP

Redes de Comunicaciones - Curso 2025/26

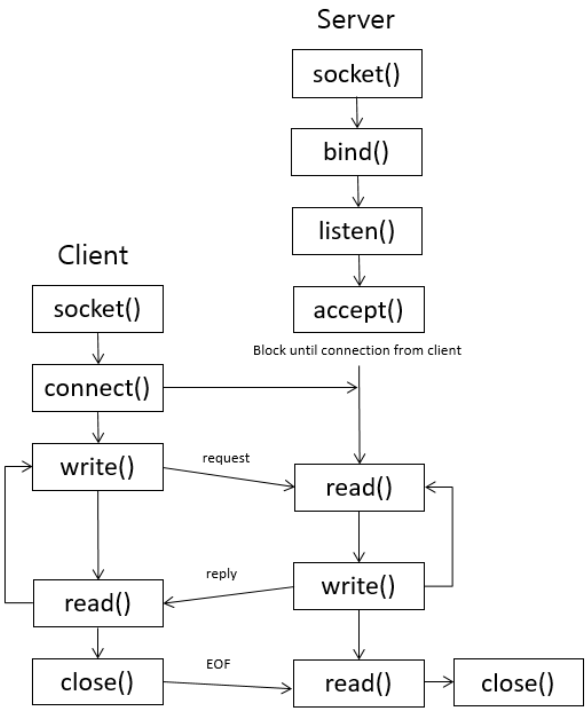
Introducción

El objetivo de este boletín de prácticas es ilustrar cómo desarrollar comunicaciones mediante *sockets* TCP haciendo uso de los principales elementos de la API de Java necesarios para llevar a cabo este fin.

En el marco del proyecto de prácticas, estos *sockets* serán necesarios a la hora de dotar de funcionalidad al programa *NanoFiles* para permitir transferir ficheros entre pares, uno actuando como servidor de ficheros y otro como cliente. Así, por un lado, el programa *NanoFiles* debe permitir ejecutar un servidor TCP a voluntad del usuario mediante el comando `serve`. Por otro lado, se debe implementar la funcionalidad del comando `peerdl`, con el fin de descargar un fichero de otros *peers*.

El ciclo de vida de los *sockets* TCP

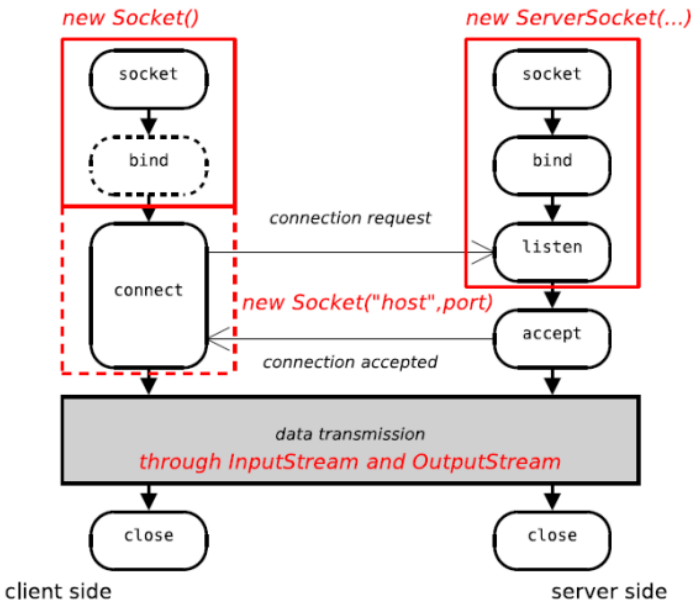
Un servidor TCP tiene un *socket* asociado a un número de puerto específico y simplemente espera a que un cliente se conecte para hacerle una petición. El cliente debe conocer el nombre del servidor (o IP) y el número de puerto por el que está escuchando. Esta figura resume el ciclo de vida tanto de los *sockets* de servidor como de cliente, a través de las llamadas al sistema que permiten la comunicación entre hosts mediante *sockets*:



Cuando el cliente solicita la conexión y es aceptada por el servidor, éste último crea un nuevo *socket* para mantener la conexión con el cliente. De esta forma, el servidor sigue escuchando por el *socket* inicial a la espera de la solicitud de conexión por parte de nuevos clientes.

Sockets TCP en Java

En la siguiente figura podemos ver, en rojo, la forma en que los constructores y métodos de las clases `Socket` y `ServerSocket` de la biblioteca de Java encapsulan dichas llamadas al sistema.



La clase `Socket` del paquete `java.net` se utiliza para crear clientes TCP, funcionalidad que en nuestro proyecto se incluye en `NFConnector`. Por ejemplo, para crear un *socket* y solicitar la conexión con un servidor basta con crear un objeto tipo `Socket` indicando la dirección del servidor como parámetro:

```
Socket socket = new Socket(serverAddress, serverPort);
```

Por otro lado, la clase `ServerSocket`, también del paquete `java.net`, se utiliza para crear servidores TCP, como es el caso del código de las clases `NFServerSimple` y `NFServer`. Tras crear un *socket* servidor, habrá que indicar mediante `bind` el puerto por el que queremos que el servidor escuche, que será el mismo puerto al que el cliente enviará la petición de conexión:

```
ServerSocket serverSocket = new ServerSocket();
serverSocket.bind(new InetSocketAddress(PORT));
```

La clase `InetSocketAddress` representa a una *dirección de socket*, es decir, IP y puerto. En el caso de no especificar una IP, se tomará la IP de la máquina donde se ejecute el servidor.

IMPORTANTE: En el caso de que ya haya un proceso escuchando en ese puerto, `bind` lanzará una excepción que debemos capturar y tratar adecuadamente.

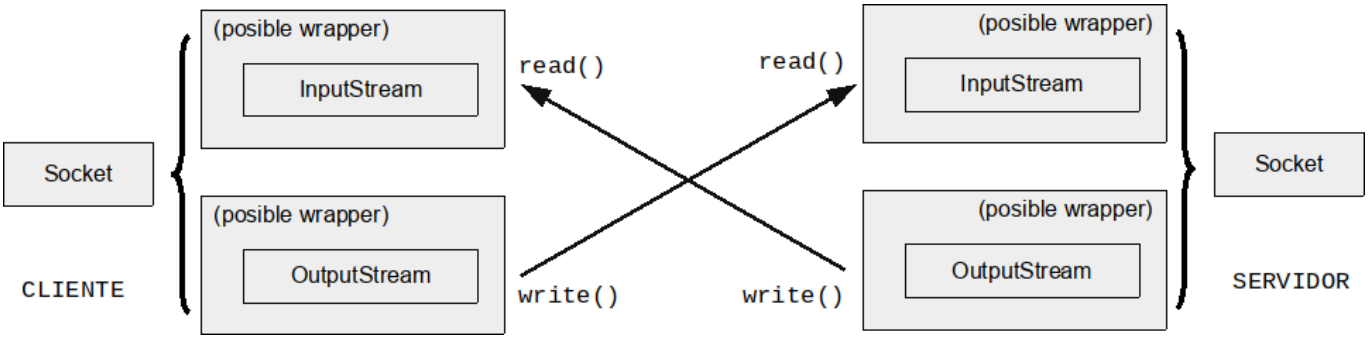
Para dejar al servidor escuchando a la espera de una conexión, bastará con llamar al método `accept()` del objeto `ServerSocket` creado y ligado previamente. En este método se quedará el servidor parado hasta que reciba una petición de conexión. En ese caso, el servidor se reanudará y el método devolverá un objeto que

representará el *socket* del cliente que le ha solicitado la conexión. Este *socket* representa un canal de comunicación **exclusivamente con el cliente** que ha iniciado la conexión.

```
Socket clientSocket = serverSocket.accept();
```

Lectura y escritura en *Sockets* mediante *Streams*.

Una vez realizada la conexión, el cliente y el servidor se comunican con los manejadores `InputStream` y `OutputStream`. Los *sockets* poseen funciones para obtener tanto el *stream* de entrada (para leer los datos que llegan al *socket*), como para obtener el *stream* de salida (para escribir los datos que se enviarán por el *socket*). La siguiente figura muestra gráficamente cuál es la relación entre los *sockets* y sus *streams*:



Hay diversas clases para manejar de diferente manera los streams de entrada y de salida. Los InputStream soportan todos ellos una lectura de un byte o de un byte[] con la función `read()`. Por su parte, todos los `OutputStream` soportan la escritura de un byte o un byte[] con la función `write()`. Sin embargo, es conveniente mencionar que a partir de los InputStream y `OutputStream` es posible hacer uso de clases más cómodas (*wrappers*) a la hora de escribir o leer datos de los sockets, como serían por ejemplo las clases `DataInputStream` o `DataOutputStream`, las cuales han sido descritas en un boletín anterior.

Por ejemplo, si queremos enviar un entero por el socket, podemos hacerlo mediante el siguiente código:

```
DataOutputStream dos = new DataOutputStream(socket.getOutputStream());
dos.writeInt(123);
```

Si sabemos que nos van a enviar un entero, podemos leerlo con el siguiente código:

```
DataInputStream dis = new DataInputStream(socket.getInputStream());
int numero = dis.readInt();
```

Es conveniente echar un vistazo a la documentación de `DataInputStream` y `DataOutputStream` para conocer todos los métodos posibles a la hora de leer/escribir de un *stream* otros tipos de datos.

Ejemplo de comunicación cliente-servidor mediante sockets TCP

Los alumnos tienen a su disposición en el Aula Virtual un proyecto Eclipse llamado `boletinTCP` a modo de ejemplo, que ilustra la comunicación entre un servidor y un cliente mediante sockets TCP.

Servidor TCP

El código de `TCPServer` ilustra el comportamiento de un **servidor TCP minimalista que atiende, a lo sumo, a un único cliente en cada momento**. Se utiliza el *socket servidor* `serverSocket` para permanecer a la escucha en el puerto 6969 a través de su método `accept`. Una vez despierte el servidor, se crea un *socket estándar* asociado a la variable `socket`, que será el extremo de la comunicación del lado servidor. A partir de este momento la conexión TCP se considera establecida.

Gracias a los manejadores de streams `dos` y `dis` es posible, respectivamente, enviar y recibir cadenas a través del canal TCP. En este ejemplo, en primer lugar se envía al cliente una cadena que previamente ha introducido el usuario por teclado, concatenada con la hora actual del sistema. Este envío se lleva a cabo a través del método `writeUTF` del manejador `dos`. A continuación, el socket se queda bloqueado a la espera de una respuesta del cliente. Este bloqueo tiene lugar en el método `readUTF` del manejador `dis` (`readUTF` es, por tanto, un método síncrono). Nótese que el blucle no volverá a ejecutar el método `accept` del *socket servidor* hasta que el cliente no conteste. Es decir, no será posible atender concurrentemente a varios clientes, sino que en caso de que haya varios, se les irá atendiendo uno a uno, en secuencia.

Cliente TCP

El cliente TCP ya sabe la IP y el puerto de escucha de su servidor. Con estos dos datos, toma la iniciativa de intentar establecer la conexión TCP, creando un *socket estándar* al que se le pasan, en el constructor, ambos datos.

Sin que medien más mensajes, la conexión se considerará establecida con el simple hecho de que funcione la creación del citado *socket*. Este socket será el extremo de la comunicación del lado cliente y, al igual que su equivalente del lado servidor, también dispondrá de los manejadores `dis` y `dos` para, respectivamente, enviar y recibir cadenas a través del canal TCP recién creado.

El resto del código implementa, en primer lugar, la espera inicial del mensaje que el servidor envía como saludo (método `readUTF` de `dis`), y seguidamente se pide una respuesta por teclado y se le envía al servidor (método `writeUTF`).

Ejercicios propuestos

Planteamos esta semana que vuestro programa *NanoFiles* permita, por un lado, que un *peer* se convierta en servidor TCP mediante el comando `serve`, y por otro, que permita mediante el comando `peerdl` que otros *peers* se conecten a un servidor que haya sido puesto a la escucha previamente.

Con el fin de que dicho objetivo general pueda realizarse como una secuencia de pasos, os sugerimos una posible forma de proceder:

1. El atributo `testModeTCP` de `NanoFiles` debería estar establecido a verdadero. Esto te permitirá probar de forma sencilla a establecer una comunicación entre un servidor y un cliente TCP que estén ejecutándose en la misma máquina, sin necesidad de contactar con el directorio. Esto te resultará útil para empezar a establecer conexiones TCP entre *peers* y programar tu protocolo de transferencia de ficheros, con independencia del grado de implementación del protocolo de comunicación con el directorio.

2. Implementa un servidor de ficheros minimalista en la clase `NFServer`, siguiendo las instrucciones que encontrarás en los *TODOs* asociados a este boletín, tomando como referencia el código del proyecto `boletinTCP` que encontrarás en el Aula Virtual, descrito anteriormente. Empieza por programar el constructor de `NFServer` y después su método `test`, el cual debería ejecutar un servidor en primer plano que escuche en el puerto 10000/tcp, capaz de atender únicamente a un cliente en cada instante.

3. Con `testModeTCP` a verdadero, al teclear el comando `serve` se ejecutará el método `test` de `NFServer` (servidor TCP). De esa forma, se ejecutará código del servidor en primer plano que has programado en el paso anterior usando el hilo principal del programa, por lo que una vez lanzado este tipo de servidor, el código no retornará al *controller*. Por tanto, ya no se aceptarán más comandos a través del shell.

4. Añade también en `NFController.processCommand()` la lógica de prueba del cliente TCP en la rama del comando `peerdl`, de forma que cuando `NanoFiles.testModeTCP` sea verdadero se invoque `controllerPeer.testTCPClient()`:

```
    case NFCommands.COM_DOWNLOAD_PEER:
        if (NanoFiles.testModeTCP) {
            controllerPeer.testTCPClient();
        } else {
            commandSucceeded =
controllerPeer.downloadFromPeers(controllerDir, targetPeerNickname,
                    targetHashSubstring);
        }
        break;
```

5. Una vez añadido el fragmento anterior, al teclear el comando `peerdl` estando activo en modo de pruebas se ejecutará el método `test` de `NFConnector` (cliente TCP). Programa el constructor de dicha clase y luego el método `test`, que debe enviar un entero cualquiera a través del socket, y luego

recibir otro entero, comprobando que el valor recibido es igual al enviado. De manera complementaria, añade el código necesario en el servidor TCP minimalista del ejercicio anterior (método `test` de `NFServer`) para primero esperar a recibir un entero a través del socket y luego enviarlo de vuelta.

6. Prueba a lanzar la aplicación *NanoFiles* e introducir el comando `serve` para convertir a ese *peer* en servidor de ficheros, escuchando en el puerto 10000/tcp. En una consola del terminal de Linux, ejecuta el siguiente comando tras lanzar el servidor: `netstat -tupan | grep "10000"`. Observa su resultado.

7. Prueba a lanzar otra instancia de la aplicación *NanoFiles* y conectarte al servidor de ficheros anterior mediante el comando `peerdl x y` (en modo de prueba, el valor de los parámetros pasados al comando se ignora). Usa el depurador para comprobar que eres capaz de enviar y recibir datos por el socket TCP, ejecutando paso a paso tanto el código del servidor de ficheros como el código cliente TCP que acabas de programar en el paso anterior. Repite el comando del *shell* de Linux del ejercicio anterior y comprueba que se ha establecido una conexión TCP entre cliente y servidor.

8. Una vez hecho esto, sustituye el envío de datos de prueba por algún mensaje binario multiformato definido en el protocolo que has diseñado. Para ello, debes hacer uso en dichos métodos del código de creación, envío y recepción de mensajes `PeerMessage` que programaste en el boletín anterior y que probaste mediante la clase `PeerMessageTest`.

> **NOTA: Recuerda desactivar `NanoFiles.testModeTCP` a partir de este momento**

9. Implementa en `NFControllerLogicP2P` los métodos `downloadFromPeers` y `downloadFileFromServers`, siguiendo los *TODO*, para cubrir la funcionalidad obligatoria de `peerdl <peer_nickname> <hash_substring>`. En esta parte, al recibir un *nickname* concreto, deberás obtener su dirección del censo de *peers* y descargar desde ese servidor. (El arranque en segundo plano del servidor con `startFileServer` se aborda en el boletín siguiente, de servidor concurrente).

10. **Mejora opcional (`peerdl *`)**: implementa `searchFilesByHash` en `DirectoryConnector`, de forma que retorne un `Map<String, InetSocketAddress[]>` donde cada clave sea un hash completo de fichero y el valor sea la lista de *peers* que lo comparten. Para construir ese mapa, parte del censo de *peers* y consulta por TCP a cada servidor si tiene un fichero cuyo hash contenga la subcadena solicitada. Después, amplía `downloadFromPeers` para que, cuando el argumento sea `*`, use dicha búsqueda y delegue la descarga en `downloadFileFromServers`.